

WT EXP - 6

Name :- Kunal Punjabi, Manav Punjabi, Hitesh rohra

Roll No :- 43,44,46

Div:- D15A

AIM:

Interfacing & Simulation of Fire and Smoke Detection System with Arduino

Interfacing Smoke and Flame Sensors with Arduino

The **MQ-2 Smoke/Gas sensor** and **IR Flame sensor** can be connected to the Arduino to detect early signs of fire or hazardous gas levels. These sensors provide either analog or digital signals that the Arduino can process and respond to using outputs like buzzers, LEDs, or even WiFi modules.

Connection Table:

Sensor Pin	Arduino Pin	Description
MQ-2 A0	A0	Analog input for smoke/gas reading
MQ-2 GND	GND	Ground
MQ-2 VCC	5V	Power supply
Flame Sensor D0	D7	Digital output for fire detection
Flame Sensor GND	GND	Ground
Flame Sensor VCC	5V	Power supply
Buzzer	D8	Output alert (buzzes on detection)
LED (optional)	D9	Visual indication of alert

Software Implementation

The Arduino IDE is used to program the board. No external libraries are required unless adding cloud or LCD support. The system continuously reads from the MQ-2 and Flame sensors, checking for:

- **High smoke levels**
- **Presence of flame**

When either is detected, the Arduino triggers a buzzer and optionally an LED for immediate alert.

Basic Program Steps:

1. **Pin Setup:** Define input and output pins in `setup()`.
2. **Sensor Monitoring:** Use `analogRead()` and `digitalRead()` in `loop()` to monitor the environment.
3. **Alert System:** If values exceed thresholds, activate buzzer and LED.
4. **Feedback:** Serial monitor logs status; optional LCD or cloud feedback possible.

Communication:

The system relies on:

- **Analog communication** for smoke level via MQ-2.
- **Digital communication** from the flame sensor.
- **UART (optional)** for connecting a WiFi module like ESP8266 to send alerts remotely.

Step-by-Step Communication Process

1. **Initializing the Sensors**
Arduino powers and prepares sensors on startup.
2. **Detecting Fire/Smoke**
Arduino continuously checks for:
 - Analog value > threshold from MQ-2
 - Digital LOW (flame detected) from IR flame sensor

3. **Processing Data**

If a dangerous level is detected, the system triggers alarms (buzzer, LED) and logs data to serial monitor or cloud.

4. **Alerting the User**

Buzzer and LED provide immediate feedback. Optional modules can send alerts to mobile or cloud dashboards.

5. **Waiting for Next Event**

The system resets after the danger has passed and continues scanning.

Sample Arduino Code:

```
int smokeA0 = A0;
int flameD0 = 7;
int buzzer = 8;
int led = 9;

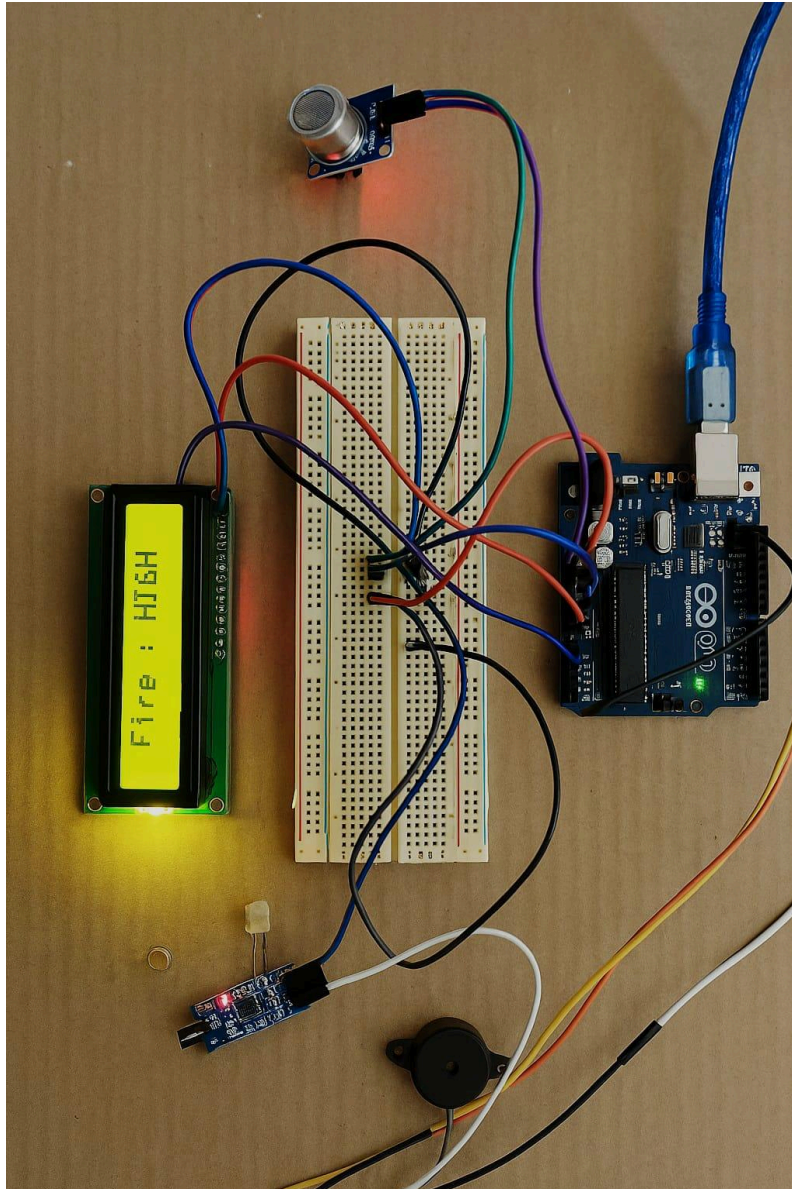
void setup() {
  pinMode(flameD0, INPUT);
  pinMode(buzzer, OUTPUT);
  pinMode(led, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int smokeLevel = analogRead(smokeA0);
  int flame = digitalRead(flameD0);

  Serial.print("Smoke Level: ");
  Serial.println(smokeLevel);

  if (smokeLevel > 300 || flame == LOW) {
    digitalWrite(buzzer, HIGH);
    digitalWrite(led, HIGH);
    Serial.println("FIRE/SMOKE DETECTED!");
  } else {
    digitalWrite(buzzer, LOW);
    digitalWrite(led, LOW);
  }

  delay(1000);
}
```



Conclusion:

In this experiment, we successfully interfaced MQ-2 and Flame Sensors with the Arduino. The system detected high levels of smoke and flame, triggering real-time alerts. It showcased how microcontrollers and sensors can be integrated for fire safety and early warning systems, demonstrating practical use in home and industrial safety solutions.